

제출물 — 3분 동선과 아키텍처

심사자가 3분 안에 네 가지를 믿을 수 있게 만드는 것이 목표다.

- 1. 게임의 한 문장: "고양이 오토배틀러인데, 보스전은 레이드다."
- 2. 플레이어의 결정이 실제로 결과를 바꾼다
- 3. 그 주장이 브라우저와 같은 로직을 쓰는 300런 하네스로 재현된다
- 4. AI는 런타임 장식이 아니라 생성 → 검증 → 시뮬레이션 → 채택/폐기 파이프라인이다

1. 3분 동선

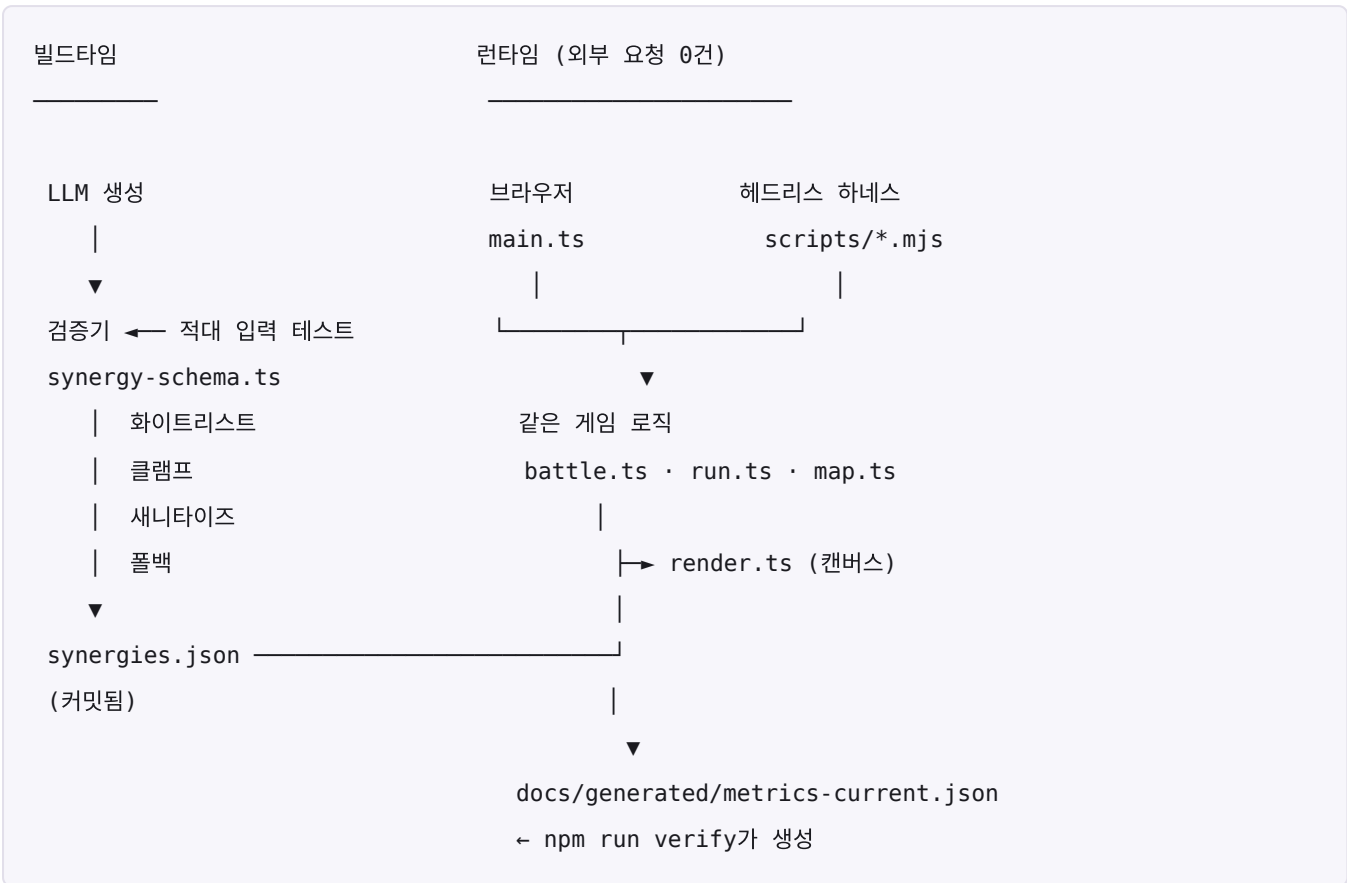
시간	보여줄 것	말할 것
0:00-0:20	게임 화면, 전환 막	한 문장 혹. "런타임 의존성 0개, 외부 요청 0건"
0:20-0:50	길을 고르고 적이 뜬 상태에서 유물 하나 사고 배치	"무엇과 싸울지 보고 산다. 조건부 보너스 + 무조건 대가 — 조건을 못 맞추면 손해만 남는다"
0:50-1:30	보스전 — 붉은 예고 흩어짐, 청록 예고 모임, 금빛 취약 창 연타	"여기가 이 게임이다. 예고를 읽고 0.5초 안에 정한다"
1:30-2:10	터미널에서 <code>npm run verify</code>	"같은 코드로 300판을 돌린다. 축이 미달이면 빨간불이 켜진다"
2:10-2:40	검증기 적대 입력 테스트 출력	"LLM 출력을 믿지 않는다. 화이트리스트·클램프·폴백을 통과한 것만 게임에 들어간다"
2:40-3:00	아키텍처 그림 + 한계	"우리가 내세우던 최대 축이 사실은 폭이었다. 다시 재서 표를 고쳤다"

1:30 구간이 이 발표의 핵심이다. 다른 제출물이 "만들었습니다"를 보여줄 때 여기서는 "틀렸을 때 어떻게 알아냈는가"를 보여준다.

영상 (60~90초, 실촬영)

- 죽은 시간 없이: 상점 한 번 → 배치 → 보스전 통째로 → 부검 화면
- 지도는 넣지 않는다. 결정 격차가 1.4웨이브로 미달이고(기준 1.5), 계측상 판 전체에서 3초밖에 안 쓴다. 3분 안에 는 값을 하는 축만 보여준다
- 보스전이 절반 이상이어야 한다. 그게 이 게임의 정체성이다
- 자막은 세 개면 충분: 붉으면 흩어져 / 청록이면 모여 / 금빛이면 때려
- AI 합성 금지(NAN 규정). 실제 플레이 화면만

2. 아키텍처



핵심은 가운데 상자 하나다. 브라우저와 하네스가 같은 `stepBattle` 을 부른다. 개입도 `state.pending` 큐로만 들어가고 판정은 전부 그 안에서 일어난다. 이 파티티가 깨지면 측정한 수치는 사람이 겪는 값이 아니게 된다.

3. 측정 결과 (재현 가능)

수치는 `docs/generated/metrics-current.md` 가 원본이다. `npm run verify` 가 생성한다.

결정 축 — 폭과 깊이를 갈라서 잰다

오래 "정책을 바꾸면 결과가 갈리는가"만 물었는데, 그렇게 재면 **축을 쓰느냐 마느냐**를 재게 된다. 그건 폭이지 깊이가 아니다. 그래서 축마다 셋을 잰다 — **바닥**(안 씀) · **뻥한 수**(생각 없이 씀) · **최선**.

폭 = **최선** - **바닥**. **깊이** = **최선** - **뻥한 수**.

축	바닥	뺀한 수	최선	폭	깊이
구매	안 삼 2.2	무작위 11.4	물빵 피벗 14.0	11.8	2.5
유물	안 삼 11.7	균형 12.2	마법사 물빵 15.0	3.3	2.8
개입	없음 51.5%	늘 탭만 95.3%	읽고 판단 96.0%	+44.5%p	+0.7%p
배치	역할 반대 6.7	자동 9.5	세로 분산 10.1	3.4	0.5
지도	—	—	—	1.4	

(개입은 첫 보스 통과율 — 모든 런이 한 번씩 맞는 유일한 칸이다)

깊이가 남는 축은 구매다. 예전에는 유물과 개입 둘이라고 적었는데, 마감 전날 조작을 손보면서 둘 다 내려갔다. 숨기지 않고 왜 그랬는지 적는다.

개입의 깊이가 사라진 것은 설계 결정의 대가다. 개입을 버튼 하나로 합쳤다 — 전에는 흠여짐과 뭉침을 다른 키로 눌러야 했고, 1.2초짜리 예고 안에서 장판 색을 읽고 **맞는 키까지 고르는** 두 가지를 동시에 요구했다. 색을 읽는 것까지는 이 게임이 요구할 만하지만 키를 고르는 것은 아니다. 버튼이 알아서 맞는 것을 고르게 하자 "읽고 판단"과 "그냥 누름"이 같은 것이 됐고, 깊이 +9.4%p가 +0.7%p로 무너졌다. **그 자리는 자원 결정이 대신 받는다** — 예고는 여섯 번 오는데 막을 수 있는 것은 둘뿐이라 "어느 것을 막을까"가 남는다. 다만 그건 아직 이 표에서 재지지 않는다. 지금 하네스가 "언제 쓰는가"를 가르는 정책을 안 갖고 있어서다.

유물이 기존 아래로 내려간 것은 버그 수정의 부작용이다. "눌러서 피했는데 데미지가 들어온다"를 계측했더니 회피가 실제로는 39%만 통했고, 세 원인을 고쳐 100%로 만들었다. 그러자 **유물이 메우던 자리가 사라졌다** — 유물 없이 가는 웨이브는 12.3에서 11.7로 그대로인데 최고 물빵이 18.1에서 15.0으로 내려왔다. 유물 보너스를 1.15~1.45배로 끌어왔지만 3.2~4.0 사이에서 오르내려 근거 있는 지점을 못 찾았고, **근거 없는 상향은 넣지 않았다.** 관문(`npm run verify`)은 이 한 축에서 빨간 채로 제출한다 — 가리는 것보다 낫다.

대신 두 축이 올라왔다. `enemyScale` 을 1.24에서 1.28로 올려 난이도를 되찾자 「궁합이 약하다 → 있다」, 「구매에 깊이가 없다 → 있다」로 바뀌었다. 구매 깊이 2.5웨이브는 기준 1.5를 넘는다.

유물은 여전히 판마다 답이 다르다. 평균 1위인 마법사 물빵도 시드의 26%에서만 최선이고, 판마다 골랐다면 19.5까지 간다(신탁 격차 4.5웨이브).

AI 파이프라인

- 시너지 규칙을 LLM이 생성 → 검증기 통과분만 `synergies.json` 으로 커밋
- 검증기 테스트가 **적대 입력**을 넣는다: 없는 트리거, 범위 초과, 태그 삽입, id 중복
- 범위 초과는 폐기가 아니라 **클램프** — 규칙을 읽지 않기 위해
- `synergies.json` 을 빈 배열로 바꿔도 게임이 돈다(폴백)

4. 한계 — 숨기지 않는다

0. 관문이 유물 한 축에서 빨간 채로 제출한다. `npm run verify` 가 exit 1이고, 이유는 유물 격차 3.3웨이브(기준 4.0)다. "눌러서 피했는데 데미지가 들어온다"를 고치면서 회피가 39% → 100%로 실제로 통하게 됐고, **유물이 메**

우던 자리가 사라졌다. 유물 보너스를 1.15~1.45배로 쓸어 봤지만 3.2~4.0 사이에서 오르내려 근거 있는 지점을 못 찾았다. 관문을 통과시키려고 기준을 낮추거나 근거 없이 상향하는 것보다, 빨간 채로 내고 왜 빨간지 적는 편이 이 프로젝트답다

1. **측정 도구 자체가 틀려 있었다 — 표를 두 번 고쳤다.** 오래 "구매 격차 11.4웨이브"를 최대의 축으로 적었는데, 폭과 깊이를 갈라 보니 그건 전부 "사느냐 마느냐"였다. 그래서 "깊은 축은 둘"로 고쳤다. **그 판정도 틀렸다** — 외부 검토가 짚어서 다시 보니 구매 정책을 전부 '카드만 보는' 것으로만 비교했고, 로스터를 읽는 정책을 놓자 깊이가 2.5로 드러났다. 개입 통과율은 보스 시도를 합산해 생존자 편향이 섞여 있었다(비개입 봇의 W6가 W3보다 높게 나온 것이 증거였다). 측정 도구 넷을 고쳤고 하나(`balance-sweep`)는 아예 죽여 있었다. **관문 안에 있던 것만 살아 있었다**
2. **지도 축이 미달이다(1.4, 기준 1.5).** 보상 강화 → 성격 분화 → 전술 슬롯 세 번을 실험했고 세 번 다 수치로 기각했다. 진단은 나와 있다 — 자원이 생선 하나뿐이라 모든 경로 선택이 "더 벌까 덜 쓸까"로 수렴한다
3. **일반 웨이브가 97.7~100%로 무해하다.** 전투 시간의 75%, 판이 끝나는 이유의 92%가 보스전이다. 일반 웨이브는 보스 사이의 호흡으로 두고 있고, 전략적이라고 주장하지 않는다
4. **개입의 "무엇을 누를까"가 없어졌다.** 버튼을 하나로 합치면서 게임이 알아서 맞는 것을 고른다 — 늘 탭만(95.3%)과 읽고 판단(96.0%)의 차이가 0.7%p다. 조작을 단순하게 만든 대가이고, 그 자리는 "예고 여섯 번 중 둘을 어디에 쓸까"가 대신 받지만 **아직 그걸 재는 정책이 하네스에 없다.** 재지 못한 것을 깊이로 주장하지 않는다
5. **취약 창 연타도 첫 보스 통과율을 3.4%p만 올린다**(회피만 92.6% → 읽고 판단 96.0%). 화면에서 가장 화려한 연출인데 결정으로서는 작다 — 회피만 해도 거의 다 넘어간다
6. **한글 비트맵 폰트 미구현.** 고유 음절 270자면 4.75KB로 들어가지만, 시너지 이름이 LLM 생성이라 닫힌 집합이 아니다. 조합형 자모(344 글리프)로 가야 하고 예셋을 구하지 못했다
7. **실시간 플레이 시간은 계측을 막 붙였다.** 이전에 쓰던 "12분"은 근거가 없었다 — 헤드리스는 웨이브 수만 세고 사람이 고민하는 시간은 안 센다. 봇으로 한 판 재 보니 **9웨이브에 166초**였고 그중 **전투가 128초(77%),** 상점 20초, 배치 6초, 지도 3초였다. 첫 보스전 진입 **19.8초,** 첫 개입 **20.6초.** 사람은 상점에서 더 오래 고민하므로 이보다 길 것이다 — **표본 20판이 쌓이기 전에는 판 길이를 공개 문서에 적지 않는다.**

`window.nyangTelemetry()` 로 실제 플레이 시간을 꺼낼 수 있다(네트워크 없음).

5. 제출 체크리스트

	항목	상태
NAN	웹 빌드 + 소스(커밋 히스토리 포함)	✓
NAN	소개 PDF	✓ docs/intro.pdf (5쪽)
NAN	AI 기술문서 PDF	✓ docs/ai-tech.pdf (10쪽)
NAN	30~60초 게임플레이 영상	✓ docs/gameplay.mp4 (52.7초 · 1280×720 · H.264+AAC · 소리 있음)
OpenAI	브라우저 플레이 링크	✓ mmporong.github.io/nyang-arena
OpenAI	16:9 썸네일	✓ docs/thumbnail.png (1280×720)
OpenAI	Codex를 개발 과정에 사용	■

스크린샷 일곱 장(docs/shots/)·썸네일·영상은 전부 배포본을 실제로 플레이해서 찍었다. `npm run docs` 가 PDF를 다시 만든다.

영상은 AI 합성이 아니다. 진짜 브라우저에서 진짜 게임이 돌고 키보드·마우스 입력이 실제로 들어간다(브라우저 자동화). 생성 모델이 만든 화면은 한 프레임도 없다. 동선은 스토리보드 그대로 — 길 고르기 → 적을 보며 구매 → 드래그 배치 → 일반 전투 → 보스 배너 → 붉은 예고 → 청록 예고 → 금빛 취약 창 → 격파. 심사 기준이 자동 조작을 문제 삼으면 같은 동선으로 손으로 다시 찍으면 된다.

남은 것은 Codex 사용 항목 하나다.

음악 4곡이 들어갔다 — 준비·보스전·타이틀·부검. 프롬프트는 docs/audio-prompts.md §3에 그대로 있고, 생성된 파일을 어떻게 측정해서 고쳤는지는 같은 문서 §7에 있다(보스곡 중역이 비었는지, 준비 → 보스 전환이 부딪히지 않는지를 dB로 확인했다).

영상에 소리가 들어 있다. 음악 트랙을 후반에 얹은 것이 아니라 브라우저가 실제로 재생한 소리를 그대로 녹음했다. 방식은 이렇다.

- **화면:** CDP `Page.startScreencast`. 브라우저가 그 페이지의 픽셀만 직접 내보낸다. X11 화면을 캡처하지 않으므로 다른 창이 섞일 수가 없다 — 화면 좌표로 찍었던 첫 시도에서 지나가던 창이 잡혀 폐기했고, 그래서 방식을 구조적으로 안전한 쪽으로 바꿨다
- **소리:** Chrome 전용 PulseAudio 싱크를 만들어 그 모니터에서만 받는다. 다른 앱 소리가 섞일 수 없다
- 화면 전송은 프레임이 바뀔 때만 오므로, 받은 5,195장을 타임스탬프 기준으로 30fps 1,559장으로 다시 골라 인코딩했다. 그냥 이어 붙이면 속도가 틀어진다

조작은 CDP로 넣었다. `?debug=1` 이 노출하는 `window.nyang` 에서 예고 색과 취약 창을 읽고 `scripts/bot-policy.mjs` 의 `makeBossBot` 과 같은 규칙으로 키를 보낸다 — 예고 하나에 한 번만 누르고(매 틱 누르면 회피 차지 6개가 첫 예고에서 다 날아간다), 취약 창에는 연타한다.

효과음 20종은 아직이다. 브라우저에서 실시간 합성하는 쪽으로 방향을 잡았고(파일 0바이트), 신호 셋이 서로 갈리는지 자동 판정하는 시제품까지 확인했다.